Task 1: Sparse Networks from Scratch

In the first task, students will train sparse networks with a fixed random connectivity mask. The mask M is generated at initialization and remains unchanged throughout training. Students should compare at least two optimization methods: SGD with momentum and Adam. The goal is to investigate how different optimizers behave when training highly sparse networks. Suggested questions include: - How does sparsity a!ect convergence speed? - How does accuracy degrade as sparsity increases? - Do different optimizers behave di!erently under high sparsity?

```
# from google.colab import drive
# drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/c
```

Load libraries

```
import os
import sys
import torch
import torch.nn as nn
import torch.optim as optim
import joblib
import torchvision
import torchvision.transforms as transforms

import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

# quarto preview 01_task_1.ipynb --to pdf
```

```
# model_path = "/content/drive/MyDrive/Posgrados/2025/Duke/ECE 685D/Final project"
# sys.path.append(model_path)

model_path = "."

from models import ResNetCIFAR
```

```
use_pin_memory = torch.cuda.is_available()

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

num_workers = min(2, os.cpu_count())
```

Load data

```
# Standard CIFAR-10 transforms
transform = transforms.Compose(
    [
        transforms.ToTensor(),
        transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)),
    ]
)

trainset = torchvision.datasets.CIFAR10(
    root=f"{model_path}/data", train=True, download=True, transform=transform
)
trainloader = torch.utils.data.DataLoader(
    trainset,
    batch_size=128,
    shuffle=True,
    num_workers=num_workers,
    pin_memory=use_pin_memory,
)

testset = torchvision.datasets.CIFAR10(
    root=f"{model_path}/data", train=False, download=True, transform=transform
)
testloader = torch.utils.data.DataLoader(
    testset,
    batch_size=128,
```

```
    shuffle=False,
    num_workers=num_workers,
    pin_memory=use_pin_memory,
)
```

Configuration

```
sparsities = [0.90, 0.95, 0.98]
optimizers = ["sgd", "adam"]
```

Main Experiment

```
epochs = 15
criterion = nn.CrossEntropyLoss()
```

```
output_dir = "/content/drive/MyDrive/Posgrados/2025/Duke/ECE 685D/Final project/results"
```

Helper Functions

```
def get_static_masks(model, sparsity):
    """Generates the fixed binary mask M"""
    masks = {}
    for name, param in model.named_parameters():
        if "conv" in name or "linear" in name:
            mask = torch.rand(param.size()).to(device)
            mask = (mask > sparsity).float()
            masks[name] = mask
        else:
            masks[name] = torch.ones(param.size()).to(device)
    return masks


@torch.no_grad()
def apply_masks(model, masks):
    """Applies W = M * theta"""
    for name, param in model.named_parameters():
        param.mul_(masks[name])
```

Actual experiment

This section runs the full experimental loop over sparsity levels and optimizers. For each configuration, a model is initialized, a fixed random mask is applied, and training proceeds while enforcing the same sparse connectivity throughout. Performance metrics are tracked at every epoch and saved for later analysis.

```
for s in sparsities:
    for opt_name in optimizers:
        print(f"\nStarting Experiment: Sparsity={s*100}%, Optimizer={opt_name}")

        # Initialize
        model = ResNetCIFAR().to(device)
        masks = get_static_masks(model, s)
        apply_masks(model, masks)

        if opt_name == "sgd":
            optimizer = optim.SGD(
                model.parameters(), lr=0.1, momentum=0.9, weight_decay=5e-4
            )
        else:
            optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=5e-4)

        history = {"train_loss": [], "train_acc": [], "test_loss": [], "test_acc": []}

        for epoch in range(epochs):
            # --- Training Phase ---
            model.train()
            train_loss, train_correct, train_total = 0.0, 0, 0

            for inputs, labels in trainloader:
                inputs, labels = inputs.to(device), labels.to(device)
                optimizer.zero_grad()
                outputs = model(inputs)
                loss = criterion(outputs, labels)
                loss.backward()
                optimizer.step()
                apply_masks(model, masks)  # Keep connectivity fixed

                train_loss += loss.item()
                _, predicted = outputs.max(1)
                train_total += labels.size(0)
                train_correct += predicted.eq(labels).sum().item()
```

```
            # --- Evaluation Phase ---
            model.eval()
            test_loss, test_correct, test_total = 0.0, 0, 0
            with torch.no_grad():
                for inputs, labels in testloader:
                    inputs, labels = inputs.to(device), labels.to(device)
                    outputs = model(inputs)
                    loss = criterion(outputs, labels)
                    test_loss += loss.item()
                    _, predicted = outputs.max(1)
                    test_total += labels.size(0)
                    test_correct += predicted.eq(labels).sum().item()

            # Record Metrics
            history["train_loss"].append(train_loss / len(trainloader))
            history["train_acc"].append(100.0 * train_correct / train_total)
            history["test_loss"].append(test_loss / len(testloader))
            history["test_acc"].append(100.0 * test_correct / test_total)

            print(
                f"Epoch {epoch+1}: Train Loss {history['train_loss'][-1]:.4f} | Test Acc {his
            )

        # --- Save Joblib ---
        experiment_results = {
            "sparsity": s,
            "optimizer": opt_name,
            "train_loss": history["train_loss"],
            "train_acc": history["train_acc"],
            "test_loss": history["test_loss"],
            "test_acc": history["test_acc"],
            "masks": masks,  # Required deliverable
            "model_state": model.state_dict(),  # For future analysis
        }

        filename = os.path.join(
            output_dir, f"task_1_{opt_name}_{int(s*100)}_sparsity.joblib"
        )
        joblib.dump(experiment_results, filename)
        print(f"Results saved to {filename}!")
```

```
Starting Experiment: Sparsity=90.0%, Optimizer=sgd
Epoch 1: Train Loss 1.4800 | Test Acc 46.15%
Epoch 2: Train Loss 1.0508 | Test Acc 57.55%
Epoch 3: Train Loss 0.8884 | Test Acc 54.54%
Epoch 4: Train Loss 0.8074 | Test Acc 51.62%
Epoch 5: Train Loss 0.7596 | Test Acc 69.40%
Epoch 6: Train Loss 0.7299 | Test Acc 64.62%
Epoch 7: Train Loss 0.7096 | Test Acc 67.02%
Epoch 8: Train Loss 0.6902 | Test Acc 68.87%
Epoch 9: Train Loss 0.6775 | Test Acc 70.44%
Epoch 10: Train Loss 0.6648 | Test Acc 69.63%
Epoch 11: Train Loss 0.6513 | Test Acc 70.86%
Epoch 12: Train Loss 0.6478 | Test Acc 60.67%
Epoch 13: Train Loss 0.6390 | Test Acc 70.60%
Epoch 14: Train Loss 0.6302 | Test Acc 71.90%
Epoch 15: Train Loss 0.6326 | Test Acc 70.14%
Results saved to /content/drive/MyDrive/Posgrados/2025/Duke/ECE 685D/Final project/results/ta

Starting Experiment: Sparsity=90.0%, Optimizer=adam
Epoch 1: Train Loss 1.5871 | Test Acc 48.50%
Epoch 2: Train Loss 1.1645 | Test Acc 58.19%
Epoch 3: Train Loss 1.0149 | Test Acc 60.58%
Epoch 4: Train Loss 0.9248 | Test Acc 66.05%
Epoch 5: Train Loss 0.8605 | Test Acc 64.21%
Epoch 6: Train Loss 0.8024 | Test Acc 67.34%
Epoch 7: Train Loss 0.7640 | Test Acc 68.56%
Epoch 8: Train Loss 0.7243 | Test Acc 70.47%
Epoch 9: Train Loss 0.6920 | Test Acc 70.43%
Epoch 10: Train Loss 0.6628 | Test Acc 70.66%
Epoch 11: Train Loss 0.6403 | Test Acc 71.42%
Epoch 12: Train Loss 0.6167 | Test Acc 71.68%
Epoch 13: Train Loss 0.5946 | Test Acc 73.01%
Epoch 14: Train Loss 0.5739 | Test Acc 72.12%
Epoch 15: Train Loss 0.5620 | Test Acc 73.34%
Results saved to /content/drive/MyDrive/Posgrados/2025/Duke/ECE 685D/Final project/results/ta

Starting Experiment: Sparsity=95.0%, Optimizer=sgd
Epoch 1: Train Loss 1.6498 | Test Acc 38.35%
Epoch 2: Train Loss 1.2002 | Test Acc 57.12%
Epoch 3: Train Loss 1.0380 | Test Acc 59.63%
Epoch 4: Train Loss 0.9561 | Test Acc 60.00%
Epoch 5: Train Loss 0.9054 | Test Acc 52.98%
```

```
Epoch 6: Train Loss 0.8657 | Test Acc 65.12%
Epoch 7: Train Loss 0.8465 | Test Acc 62.64%
Epoch 8: Train Loss 0.8222 | Test Acc 63.60%
Epoch 9: Train Loss 0.8056 | Test Acc 65.55%
Epoch 10: Train Loss 0.7962 | Test Acc 67.65%
Epoch 11: Train Loss 0.7894 | Test Acc 66.20%
Epoch 12: Train Loss 0.7805 | Test Acc 55.36%
Epoch 13: Train Loss 0.7666 | Test Acc 60.81%
Epoch 14: Train Loss 0.7665 | Test Acc 67.60%
Epoch 15: Train Loss 0.7544 | Test Acc 61.46%
Results saved to /content/drive/MyDrive/Posgrados/2025/Duke/ECE 685D/Final project/results/ta

Starting Experiment: Sparsity=95.0%, Optimizer=adam
Epoch 1: Train Loss 1.8323 | Test Acc 36.19%
Epoch 2: Train Loss 1.5417 | Test Acc 41.56%
Epoch 3: Train Loss 1.4106 | Test Acc 42.55%
Epoch 4: Train Loss 1.3250 | Test Acc 45.08%
Epoch 5: Train Loss 1.2622 | Test Acc 48.91%
Epoch 6: Train Loss 1.2182 | Test Acc 49.05%
Epoch 7: Train Loss 1.1826 | Test Acc 50.83%
Epoch 8: Train Loss 1.1545 | Test Acc 54.83%
Epoch 9: Train Loss 1.1285 | Test Acc 51.85%
Epoch 10: Train Loss 1.1068 | Test Acc 56.14%
Epoch 11: Train Loss 1.0861 | Test Acc 56.15%
Epoch 12: Train Loss 1.0698 | Test Acc 57.18%
Epoch 13: Train Loss 1.0574 | Test Acc 55.74%
Epoch 14: Train Loss 1.0411 | Test Acc 57.95%
Epoch 15: Train Loss 1.0289 | Test Acc 57.92%
Results saved to /content/drive/MyDrive/Posgrados/2025/Duke/ECE 685D/Final project/results/ta

Starting Experiment: Sparsity=98.0%, Optimizer=sgd
Epoch 1: Train Loss 1.7946 | Test Acc 36.59%
Epoch 2: Train Loss 1.5153 | Test Acc 46.35%
Epoch 3: Train Loss 1.4071 | Test Acc 45.13%
Epoch 4: Train Loss 1.3515 | Test Acc 42.58%
Epoch 5: Train Loss 1.3171 | Test Acc 45.01%
Epoch 6: Train Loss 1.2975 | Test Acc 42.05%
Epoch 7: Train Loss 1.2746 | Test Acc 47.86%
Epoch 8: Train Loss 1.2583 | Test Acc 50.42%
Epoch 9: Train Loss 1.2576 | Test Acc 44.14%
Epoch 10: Train Loss 1.2435 | Test Acc 50.08%
Epoch 11: Train Loss 1.2302 | Test Acc 46.97%
Epoch 12: Train Loss 1.2275 | Test Acc 46.45%
```

```
Epoch 13: Train Loss 1.2237 | Test Acc 44.64%
Epoch 14: Train Loss 1.2136 | Test Acc 36.23%
Epoch 15: Train Loss 1.2156 | Test Acc 50.50%
Results saved to /content/drive/MyDrive/Posgrados/2025/Duke/ECE 685D/Final project/results/ta

Starting Experiment: Sparsity=98.0%, Optimizer=adam
Epoch 1: Train Loss 1.9828 | Test Acc 30.40%
Epoch 2: Train Loss 1.7337 | Test Acc 37.47%
Epoch 3: Train Loss 1.6010 | Test Acc 40.73%
Epoch 4: Train Loss 1.5292 | Test Acc 41.95%
Epoch 5: Train Loss 1.4710 | Test Acc 43.88%
Epoch 6: Train Loss 1.4234 | Test Acc 45.62%
Epoch 7: Train Loss 1.3807 | Test Acc 45.25%
Epoch 8: Train Loss 1.3495 | Test Acc 45.60%
Epoch 9: Train Loss 1.3253 | Test Acc 46.64%
Epoch 10: Train Loss 1.3034 | Test Acc 47.59%
Epoch 11: Train Loss 1.2862 | Test Acc 48.19%
Epoch 12: Train Loss 1.2712 | Test Acc 49.35%
Epoch 13: Train Loss 1.2562 | Test Acc 48.64%
Epoch 14: Train Loss 1.2407 | Test Acc 46.10%
Epoch 15: Train Loss 1.2298 | Test Acc 47.28%
Results saved to /content/drive/MyDrive/Posgrados/2025/Duke/ECE 685D/Final project/results/ta
```

The printed outputs provide a quick view of training loss and test accuracy across epochs for each configuration. As sparsity increases, performance generally degrades, reflecting the reduced capacity of the network. Differences between optimizers also become more pronounced at higher sparsities, with Adam often converging more smoothly than SGD.

These logs only show a subset of the recorded metrics. Additional information such as training accuracy, test loss, and the full training history are stored in the saved joblib files for more detailed analysis.

Evaluation

Functions

```python
def plot_training_and_validation(results):
    sns.set_theme(style="whitegrid", context="notebook", palette="deep")

    sparsities = [0.90, 0.95, 0.98]
    opts = ["sgd", "adam"]
```

```
    for s in sparsities:
        rows = []

        # --- Reshape data into tidy format ---
        for opt in opts:
            if s in results[opt]:
                n_epochs = len(results[opt][s]["train_loss"])
                for epoch in range(n_epochs):
                    rows.append(
                        {
                            "epoch": epoch,
                            "optimizer": opt.upper(),
                            "split": "Train",
                            "loss": results[opt][s]["train_loss"][epoch],
                            "accuracy": results[opt][s]["train_acc"][epoch],
                        }
                    )
                    rows.append(
                        {
                            "epoch": epoch,
                            "optimizer": opt.upper(),
                            "split": "Test",
                            "loss": results[opt][s]["test_loss"][epoch],
                            "accuracy": results[opt][s]["test_acc"][epoch],
                        }
                    )

        df = pd.DataFrame(rows)

        # --- Create 2x2 figure ---
        fig, axes = plt.subplots(2, 2, figsize=(14, 10), sharex=True)
        fig.suptitle(
            f"Optimization Analysis at {int(s*100)}% Sparsity",
            fontsize=18,
            weight="bold",
        )

        # --- Train Loss ---
        sns.lineplot(
            data=df[df["split"] == "Train"],
            x="epoch",
            y="loss",
```

```
        hue="optimizer",
        linewidth=2.5,
        ax=axes[0, 0],
    )
    axes[0, 0].set_title("Train Loss")
    axes[0, 0].set_ylabel("Loss")
    axes[0, 0].legend(title="")

    # --- Test Loss ---
    sns.lineplot(
        data=df[df["split"] == "Test"],
        x="epoch",
        y="loss",
        hue="optimizer",
        linewidth=2.5,
        ax=axes[0, 1],
    )
    axes[0, 1].set_title("Test Loss")
    axes[0, 1].set_ylabel("Loss")
    axes[0, 1].legend(title="")

    # --- Train Accuracy ---
    sns.lineplot(
        data=df[df["split"] == "Train"],
        x="epoch",
        y="accuracy",
        hue="optimizer",
        linewidth=2.5,
        ax=axes[1, 0],
    )
    axes[1, 0].set_title("Train Accuracy")
    axes[1, 0].set_xlabel("Epoch")
    axes[1, 0].set_ylabel("Accuracy (%)")
    axes[1, 0].legend(title="")

    # --- Test Accuracy ---
    sns.lineplot(
        data=df[df["split"] == "Test"],
        x="epoch",
        y="accuracy",
        hue="optimizer",
        linewidth=2.5,
```

```
            ax=axes[1, 1],
        )
        axes[1, 1].set_title(
            "Test Accuracy",
        )
        axes[1, 1].set_xlabel("Epoch")
        axes[1, 1].set_ylabel("Accuracy (%)")
        axes[1, 1].legend(title="")

        # --- Final polish ---
        for ax in axes.flat:
            ax.grid(True, alpha=0.3)

        sns.despine()
        plt.tight_layout()

        plt.savefig(f"img/sparsity_{int(s*100)}.png", dpi=300, bbox_inches="tight")
        plt.show()
```

Read results

```
output_dir = "results"
```

```
# Patch torch.load to force CPU
_original_torch_load = torch.load


def cpu_torch_load(*args, **kwargs):
    kwargs["map_location"] = torch.device("cpu")
    return _original_torch_load(*args, **kwargs)


torch.load = cpu_torch_load

results = {}
```

```
for opt_name in optimizers:
    results[opt_name] = {}
    for s in sparsities:
        file_path = os.path.join(
```

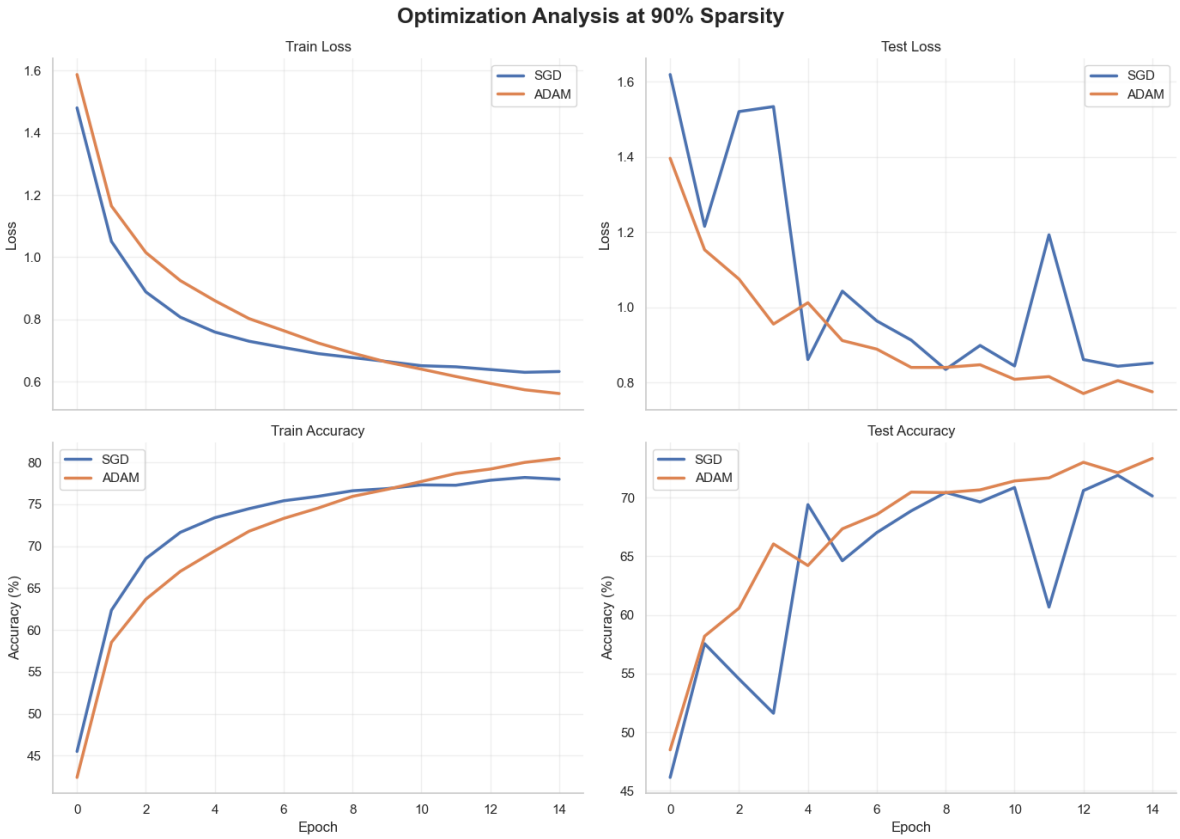
```
            output_dir, f"task_1_{opt_name}_{int(s*100)}_sparsity.joblib"
        )

        if os.path.exists(file_path):
            data = joblib.load(file_path)
            results[opt_name][s] = data
            print(f"Loaded: {file_path}")
        else:
            print(f"Missing: {file_path}")
```

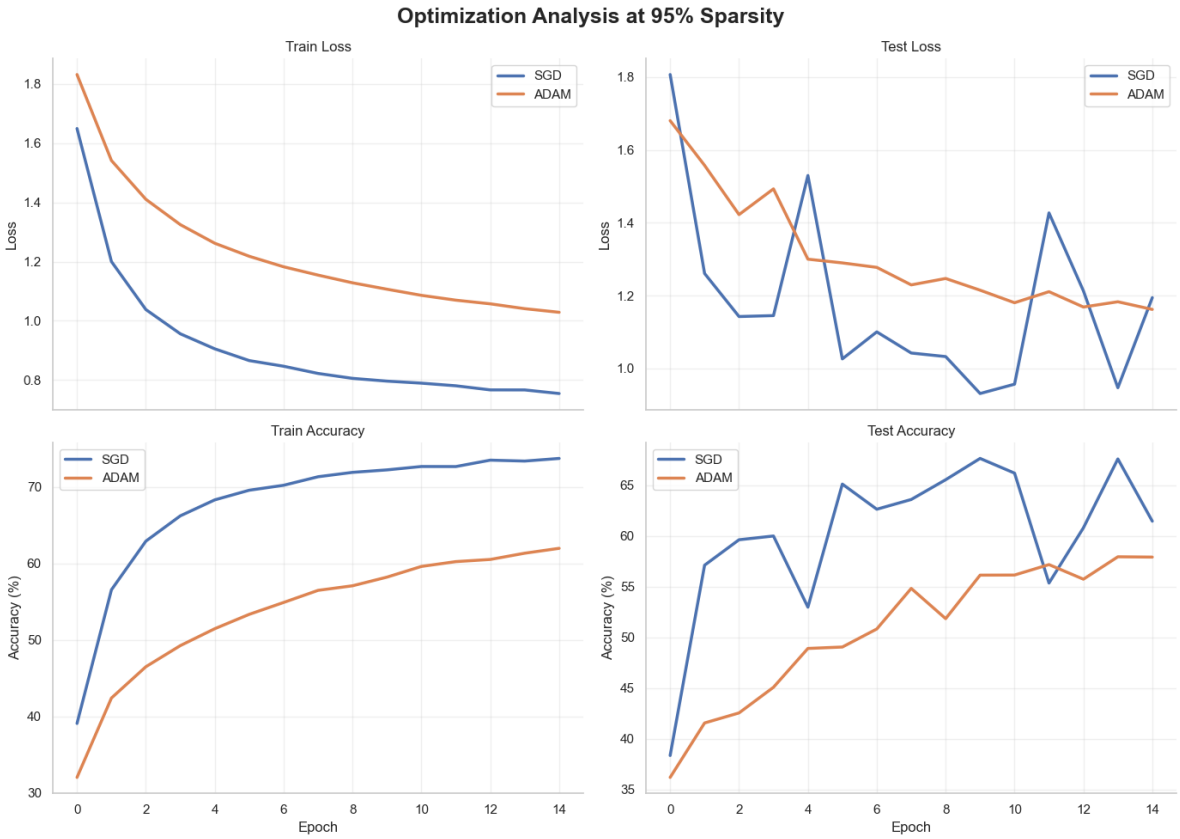
```
Loaded: results/task_1_sgd_90_sparsity.joblib
Loaded: results/task_1_sgd_95_sparsity.joblib
Loaded: results/task_1_sgd_98_sparsity.joblib
Loaded: results/task_1_adam_90_sparsity.joblib
Loaded: results/task_1_adam_95_sparsity.joblib
Loaded: results/task_1_adam_98_sparsity.joblib
```

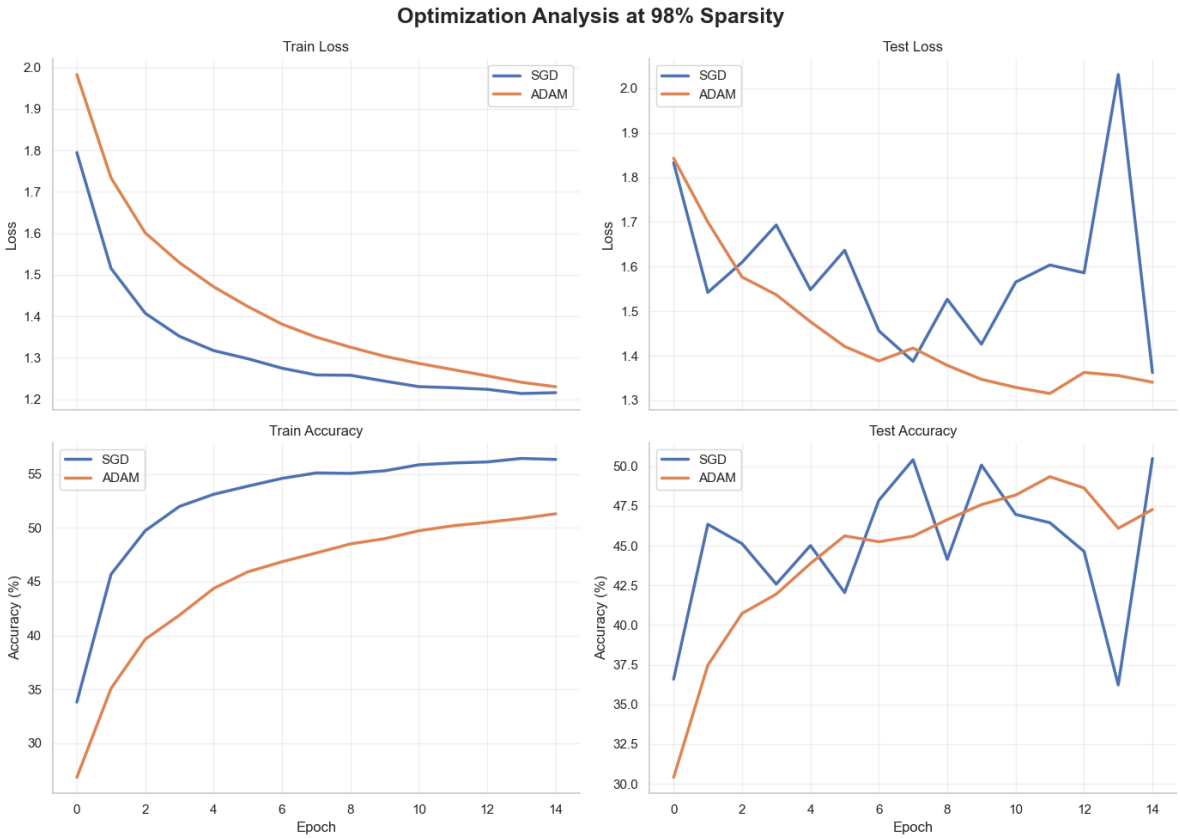
Analysis

Training and Validation Curves

```
plot_training_and_validation(results)
```


Optimization Analysis at 90% Sparsity
Train Loss
Test Loss
Train Accuracy
Test Accuracy
SGD
ADAM
Loss
Accuracy (%)
Epoch


Optimization Analysis at 95% Sparsity
Train Loss
Test Loss
Train Accuracy
Test Accuracy
SGD
ADAM
Loss
Accuracy (%)
Epoch


Optimization Analysis at 98% Sparsity
Train Loss
Test Loss
Train Accuracy
Test Accuracy
SGD
ADAM
Loss
Accuracy (%)
Epoch